

Disponemos de un conjunto de clases empaquetadas dentro de un paquete del proyecto. **Analiza** el siguiente código para ejecutar clases Java del mismo paquete y responde razonadamente a las siguientes cuestiones en un archivo **Ejercicio2.pdf**:

```
import java.io.File;
import java.io.IOException;

public final class ProcesoJava {

    public static Process exec(Class clase) throws IOException {
        String javaHome = System.getProperty("java.home");
        String javaBin = javaHome + File.separator + "bin" + File.separator + "java";
        String classpath = System.getProperty("java.class.path");
        String className = clase.getCanonicalName();

        ProcessBuilder builder = new ProcessBuilder(javaBin, "-cp", classpath, className);
        return builder.start();
    }
}
```

1. ¿Qué tipo de excepción se lanza cuando el proceso falla en su ejecución?

IOException se lanza. IndexOutOfBoundsException, NullPointerException, SecurityException

2. Busca información sobre **System.getProperty()** ¿qué propiedad del sistema se usa para obtener el sistema operativo en que se ejecuta la JVM?

```
static String      getProperty(String key)
                  Gets the system property indicated by the specified key.
```

System.getProperty(String key)

Permite acceder a las propiedades del sistema y obtener información sobre el entorno de ejecución.

Devuelve una String que es una propiedad de sistema indicada por la key(clave)

Claves disponibles:

Sistema Operativo:

- os.name → Esta propiedad se usa para saber en que sistema operativo está ejecutando la JVM (windows ,Linux)
- os.version
- os.arch
- user.home
- user.dir
- user.name
- file.separator
- path.separator
- line.separator

Java:

- java.version
- java.vendor
- java.home

```
static String      getProperty(String key, String def)  
                  Gets the system property indicated by the specified key.
```

System.getProperty(String key, String def)

Es la versión sobrecargada que incluye un valor por defecto (def - default).

Comportamiento:

Si la propiedad existe → retorna el valor de la propiedad

Si la propiedad NO existe → retorna el valor por defecto (def)

Nunca retorna null (a diferencia de la versión sin valor por defecto)

3. ¿Para qué nos sirve getCanonicalName() en su uso del lanzamiento de procesos Java?

Cuando necesite ejecutar una clase Java específica desde la línea de comandos.

4. ¿Cuántas JVM existirán en el sistema operativo si el proceso padre ejecuta sin fallos el siguiente código, siendo P1 y P2 clases del mismo paquete que ProcesoJava?

```
ProcesoJava.exec(P1.class);  
ProcesoJava.exec(P1.class);  
ProcesoJava.exec(P2.class);
```

Existirán 3 instancias de JVM Son 4 siendo el padre 1 de ellas

5. ¿En qué estado queda el proceso padre tras ejecutar el código anterior? ¿Puede que los subprocesos se encuentren TODOS en estado de ejecución?

El proceso padre puede terminar inmediatamente o esperar a los hijos con waitFor()

Los subprocesos pueden estar todos en ejecución si no se utiliza waitFor()

Solo puede haber 1 solo proceso en ejecución.

6. Comprueba en la lista de procesos de sistema operativo si se lanza una nueva JVM al ejecutarse el método estático ProcesoJava.exec.